

Phase 1 PM Onboarding — Work Summary for Mentor Review

Sep 25, 2026 · @Chahat

About this document

This is a summary of my Phase 1 Product Manager onboarding work from 7 to 25 September 2026, written so you can follow it without having seen my files. For every module it explains **why** the module exists, **how** I worked through it, **what** I produced, and **what I learned** step by step, including the mistakes I made and how I corrected them.

The headline: I have completed **38 of 76 tasks** (Modules 0–4 in full, plus Tasks 5.1–5.3 of Module 5). That's about **104 of the 225 estimated hours**, and I've logged roughly **91 hours** myself.

How to read it

- Sections 1–3 give context: how the plan is structured, where I am, and the working habits I set up.
- Sections 4–9 go module by module. Each follows the same pattern: *Why this module* · *How I worked* · *What I produced* · *What I learned*.
- Section 10 covers the patterns I've noticed in my own mistakes, and Section 11 lists my questions for you and a glossary of the terms used.

Where the evidence lives: my onboarding workbook (`PM_Onboarding.xlsx` : task tracker, daily journal, prompt library), the task write-ups in my `01-Journal/Claude outputs` folder, and the Claude Design prototypes referenced in Module 4.

1. The onboarding plan

Phase 1: Foundations is a structured plan of 225 hours across 9 modules and 76 tasks for a PM joining straight from university. It builds three skills together: working out **what's worth building** (discovery and product thinking), **getting it built** (delivery), and **working fluently with Claude** (Chat, Cowork, Claude Design). Figma is deliberately left out; prototyping is done in Claude Design.

flowchart LR

```

M0["M0 Landing Well"] --> M1["M1 Claude Foundations"]
M1 --> M2["M2 Discovery<br/>PawPal"]
M2 --> M3["M3 Working with<br/>Designers"]
M3 --> M4["M4 Idea to Screen<br/>Harvest Hub"]
M4 --> M5["M5 Delivery<br/>in progress"]
M5 --> M6["M6 Product Sense<br/>and AI Patterns"]
M6 --> M7["M7 Cowork and<br/>Scaling Yourself"]
M7 --> M8["M8 Capstone<br/>ClaimLens"]

```

Modules 0–2 are the first fortnight, Modules 3–4 bridge into design and engineering, Module 5 is the operational core, and Modules 6–8 bring everything together in a capstone.

How a task works. Every task has a type (Read, Set up, Build, Learn, Practise, Present or Reflect), an hour estimate, a "why it matters" note, resources, and five exercise steps. Most tasks end in an **artefact**, something I made, such as a critique, a spec or a prototype. I record each step's status, dates and hours in the Task Tracker, and a Dashboard sheet calculates progress automatically.

The three practice briefs. These are fictional client requests. Each deliberately leaves things out, so part of the exercise is noticing what's missing.

Brief	The client's request, in short	What it deliberately leaves out	Used in
A • PawPal	A small Ahmedabad team wants a pet-sitting app for India. Main worry: trust. Wants reviews, "some kind of verification", booking, messaging and UPI payments. Small budget, four months.	Who the sitters are, what "verification" means, what happens when things go wrong, boarding vs home visits, how both sides of the marketplace get started	Module 2

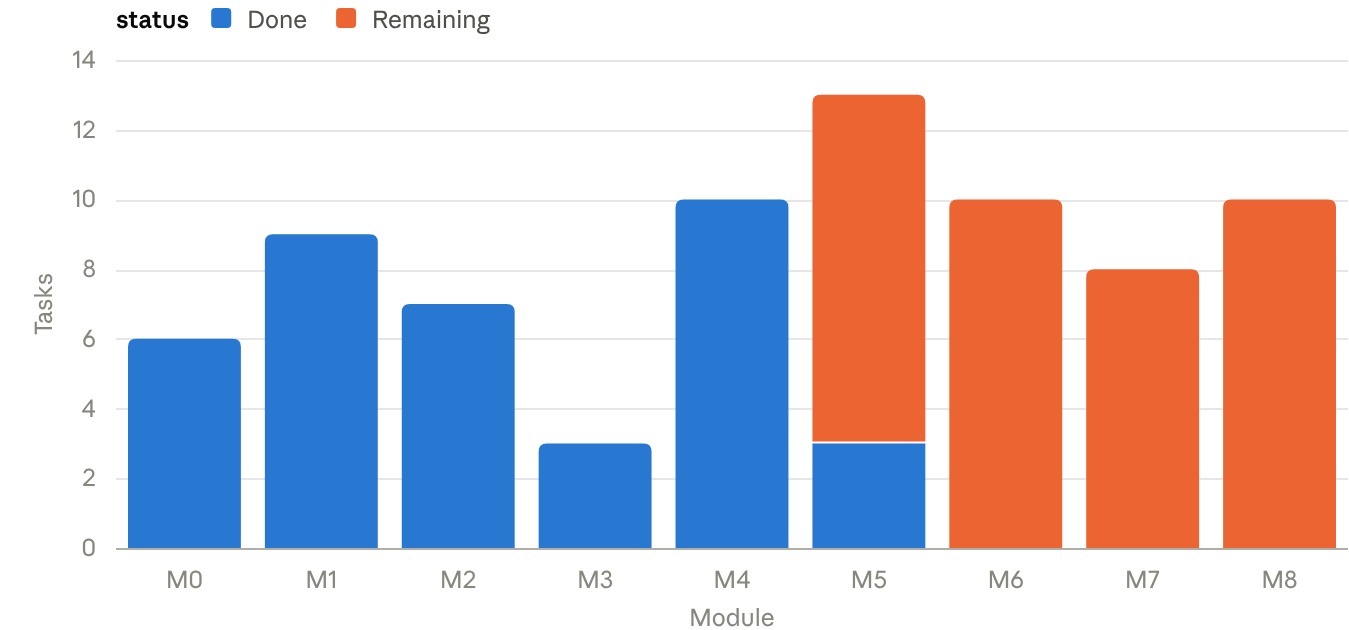
Brief	The client's request, in short	What it deliberately leaves out	Used in
B · Harvest Hub	A produce distributor serving ~200 restaurants and grocers in 3 cities runs on WhatsApp and a spreadsheet. Needs an internal admin system for orders, stock, driver assignment and late tracking. A late order is often worthless.	How orders arrive, what drivers need, what "how the business is doing" means, existing systems, stock running out mid-order	Modules 4–6
C · ClaimLens	An insurer handling ~4,000 motor claims a month wants an AI-assisted triage console with plain-language search. Adjusters are sceptical, and every decision must be auditable.	What "unusual" means, what happens when the AI is wrong, audit trail contents, how the tool is introduced to a sceptical team	Modules 6, 8

A standing habit: fifteen minutes at the end of each day in a learning journal, answering what I made, what confused me, what I want to ask, and what I'd do differently.

2. Progress at a glance

Modules 0–4 are complete, Module 5 is underway, and Modules 6–8 haven't started.

Tasks done by module



38 of 76 tasks complete as of 25 Sep 2026

PM_Onboarding.xlsx Task Tracker · 9 modules

Module	Estimated hrs	Hours I logged	Note
0 · Landing Well	16	27.9	The policy reading alone took ~20 hrs against a 6-hr estimate
1 · Claude Foundations	22	15.7	
2 · Discovery	20	10.6	
3 · Working with Designers	8	6.1	
4 · From Idea to Screen	31	23.3	A few steps have no hours logged
5 · Delivery (3 of 13 tasks)	40	7.5	In progress

Where I am now: Task 5.4 Jira, hands on (I've watched the tutorial and set up a sandbox; next I build the Harvest Hub board) and Task 5.5 Testing and QA (the reading is done).

3. My working system

Along the way I built a set of habits and reusable tools. They're pinned at the top of my journal and in my prompt library, and they explain a lot about how I did the later work.

Tool or rule	What it is	Why it exists	Where it came from
Learning journal	A Claude Project with a daily entry: what I made, what confused me, what I want to ask, what I'd do differently. Its instructions tell Claude to ask one sharp question about my confusion instead of reassuring me.	Makes confusion visible while it's fresh, and gives me a record to compare against later	Task 0.3
30-minute rule	Try on my own for 30 minutes (check resources, test my best guess), then ask.	Stops me both spinning for hours and asking too early	Task 0.6
Blocker template	Urgency · goal · what I've tried · what happened · what I think the issue is and what I've ruled out · attachment · the specific help I need	A senior colleague can unblock me in two minutes instead of twenty. Version 2 was tightened after Claude stress-tested version 1.	Task 0.6
Prompt skeleton	Role · Context · Task · Format · Constraints, used as a checklist before writing a prompt	Turns prompting from guesswork into a repeatable method	Task 1.2
Verification rule	Always check medical claims, education-industry claims, business numbers, dates, locations and citations before trusting them.	AI can be confidently wrong, so checking is part of the job	Task 1.8

Tool or rule	What it is	Why it exists	Where it came from
Fidelity rule	Name the one uncertainty I'm trying to resolve, then pick the cheapest prototype that actually tests it.	Stops me over-building or testing the wrong thing	Task 2.5
Five screen checks	Hierarchy, spacing, alignment, contrast, obvious primary action	A shared vocabulary for giving designers feedback they can act on	Task 3.1
Protect the core requirement	In multi-round prototype work, restate the one requirement that must survive, and check it at the end of every round.	I watched a requirement quietly erode over 10 rounds (Module 4)	Task 4.6
Prompt library	About 30 reusable prompts grouped by purpose (Learn, Critique, Ideate, Write, Analyse, Review), each with a "use this when" line and [BRACKETED] placeholders	So good prompts compound instead of being rewritten each time	Task 1.9, still growing

4. Module 0 • Landing Well (6 tasks, complete)

Why: at work, most rules aren't written down the way they were at university. This module makes the unwritten parts explicit: policies, tools, how to be transparent, ethics, and how to ask for help.

How I worked, and what I produced

Task	What I did	What I produced
0.1 Company policies	Read all 15 policies twice, listed unfamiliar terms and took my questions to you	Written answers to five realistic policy situations

Task	What I did	What I produced
0.2 Tools and workspace	Set up my folder structure and confirmed which tools I have now (Cowork, Claude Design) and later (Jira, internal Skills)	A working setup
0.3 Learning journal	Created the journal Project with its "ask me one sharp question" instruction	Daily journal entries
0.4 Transparency	Wrote three situations where speaking up takes courage, drafted the exact messages, and had Claude critique them	Before-and-after message pairs
0.5 Ethics	Found three dark patterns (manipulative design) in Zomato, redesigned one, and debated both sides with Claude	A dark-pattern analysis and redesign
0.6 Asking good questions	Learned about the "XY problem" (asking about my attempted fix instead of my actual problem)	The 30-minute rule and blocker template

What I learned, step by step

- 1. Applying policies, not just reading them.** For example: never share one client's work with another client; strip client identity and business numbers from anything I'd put in a portfolio; use the VPN on public wifi; tell HR directly first for short-notice leave; don't give outsiders feature specifics and point them to PR.
- 2. Check permissions before calling something missing.** I reported that a Claude setting "didn't exist". My journal's first sharp question asked whether it was actually an org-admin setting I don't have access to. Lesson: check permission levels before assuming a feature is missing.
- 3. Transparency means sharing early.** My first drafts were long and apologetic. The rewrites follow one pattern: what I noticed → its specific consequence → what I've already done to contain it → a specific ask → by when.
- 4. Persuasion vs manipulation.** Zomato's patterns: free-delivery tags tied to hidden thresholds, fees that come back even when delivery is free, and Gold-membership prompts shown right after I already qualify for free delivery. My redesign shows the

Gold prompt only to users who are actually ineligible. In the debate, the “it’s just normal upsell timing” argument was harder to counter than I expected.

5. Module 1 · Claude Foundations (9 tasks, complete)

Why: everything later in the plan depends on getting reliably good work out of Claude, and on knowing when not to trust it. The module moves from vague one-line prompts to structured ones, and builds the habit of checking what comes back.

How I worked, and what I produced

Task	What I did	What I produced
1.1 What an LLM is	Wrote a five-sentence explanation of how a language model works and had Claude correct it	Corrected explanation and my three biggest surprises
1.2 Anatomy of a prompt	Rewrote three lazy prompts (“write about design systems”, “what is automation?”, “what are tickets?”) using the skeleton, then ran both versions side by side	Three annotated comparisons, five differences each
1.3 Prompting techniques	Wrote four versions of a “summarise this meeting into action items” prompt (bare, with examples, XML-tagged, step-by-step) and ran them on the same fictional notes	A ranked comparison
1.4 Debugging a prompt	Took a hard ask (onboarding copy for a pet-sitting app for anxious first-time users) through five rounds, changing one thing per round	A five-round log with a diagnosis for each round

Task	What I did	What I produced
1.5 What Chat can do	Tested six features (attachments, web search, artifacts, extended thinking, styles, memory) and checked each claim against the Help Center	A one-page cheat sheet
1.6 Projects	Set up three Projects with different instructions and files, then asked each the same question	A comparison of the answers
1.7 Tutor, not oracle	Explained "why PMs use Jira" to Claude and let it question me until my explanation held up	Reusable Tutor and Quiz Me prompts
1.8 Verification	Asked five checkable factual questions and verified each against a primary source	My verification rule
1.9 Prompt library	Collected and organised everything into a library, then acted on Claude's review	A 27-entry library, since grown to ~30

The debugging loop from Task 1.4

flowchart LR

```

A["Write judging criteria<br/>before reading output"] --> B["Run prompt"]
B --> C["Find the single<br/>biggest gap"]
C --> D["Diagnose: context,<br/>constraint, format, task?"]
D --> E["Change ONE thing"]
E --> B

```

Across the five rounds I added: a trust signal (missing constraint), calm positive language (missing constraint), no anxiety-triggering openers (missing constraint), a brand-values line (missing context), and a question as the opening line (wrong format).

What I learned, step by step

- 1. Models write one token at a time.** A token is a small chunk of text. Each new token is predicted from everything before it. So a model's "confidence" measures how plausible the wording sounds, not whether it's true, and once it commits to a wrong detail it keeps building on it. That's why AI can be fluently wrong.

2. **Structure beats length.** The skeleton versions won on five things every time: a clear audience, specific content, structure, bounded length and plain language. The lazy "what are tickets?" prompt even answered the wrong question: it listed ticket fields instead of explaining the lifecycle.
3. **More technique isn't automatically better.** The plain prompt ranked first. My "with examples" prompt failed because a single hard-coded example only matches that one exact case; examples need to show a repeatable pattern.
4. **Change one variable at a time.** Most of my fixes were missing rules rather than missing facts. I also learned to write judging criteria before reading the output, which I got wrong the first time.
5. **Instructions shape reasoning; files ground it in facts.** The three Projects reasoned differently (business trade-offs vs UX principles vs my own project history) but reached the same recommendation.
6. **Being questioned exposes what I only think I understand.** Under questioning I admitted I didn't know the name of Jira's main view (the *board*) because I hadn't used it. My final explanation: Jira is where decided work gets tracked as tickets on a roadmap; brainstorming happens before Jira.
7. **Correct can still be context-dependent.** All five factual answers checked out, but two depended on interpretation (the "ideal" BMI range varies by guideline). That shaped which categories went into my verification rule.

6. Module 2 · Discovery and Problem Definition (7 tasks, complete)

Why: products start with a person who has a problem, described imperfectly by a client. This module teaches the loop for getting from a vague brief to a problem statement a team can build against. I ran it end to end on **Brief A (PawPal)**.

The method: design thinking in five stages

flowchart LR

```
E["1 Empathise<br/>understand the user"] --> D["2 Define<br/>one problem statement"]
```

```
D --> I["3 Ideate<br/>many ideas"]
```

```
I --> P["4 Prototype<br/>cheapest test"]
```

```
P --> T["5 Test<br/>watch real users"]
```

```
T -. new insight .-> E
```

```
T -. wrong problem .-> D
```

```
P -. idea fails .-> I
```

The dotted arrows are the backward loops: testing often sends you back to an earlier stage. When I thought about which stage I'd be most tempted to skip, the answer was **Define**, because it's easy to believe the problem is already clear when it isn't.

How I worked through each stage on PawPal

flowchart TD

```
A["10 interview questions<br/>about past behaviour"] --> B["15-min interview<br/>with a real pet owner"]
B --> C["Cluster notes<br/>into themes"]
C --> D["POV statement<br/>3 drafts"]
D --> E["5 How Might We<br/>questions"]
E --> F["8 own sketches +<br/>25 Claude ideas"]
F --> G["7 themes →<br/>3 directions"]
G --> H["Pick fidelity<br/>per direction"]
H --> I["Usability script +<br/>5-screen wireframe"]
I --> J["Test with<br/>a colleague"]
```

Stage	What I did	Key output
Empathise (2.2)	Wrote ten interview questions and revised them so nine ask about past behaviour ("tell me about the last time...") rather than hypotheticals, which get idealised answers. Interviewed a colleague who owns a golden retriever, Bodhi.	Surprising finding: in 5.5 years the family never left Bodhi with anyone outside the family. Even a paid dog walker only worked when the owner stayed nearby. During a medical emergency they searched for boarding and found nothing they trusted.
Define (2.3)	Clustered one observation per line into themes, then wrote a point-of-view (POV) statement in the form <i>[user] needs [need] because [insight]</i> , followed by How Might We (HMW) questions: problems reframed as open questions.	Final POV: the owner needs confidence that his pet will feel secure with a caregiver even when he isn't there, because vetting and payment alone didn't transfer the trust he relies on.
Ideate (2.4)	Did Crazy Eights (eight sketches in eight minutes), asked Claude for 25 divergent ideas, merged them into seven themes and chose three.	Three directions: Recreate the Owner's Presence · Gradual Trust-Building Protocols · Caregiver Vetting & Matching for Calming Ability

Stage	What I did	Key output
Prototype (2.5)	Compared four fidelity levels (paper, wireframe, clickable, hi-fi) by the question each one answers, then chose one per direction.	Paper for the first two directions, a wireframe for vetting and matching, plus my fidelity rule
Test (2.6)	Wrote five goal-based test tasks ("find a sitter within ₹500/day", not "click Search"), built a five-screen wireframe and watched a colleague try the tasks.	Real failure found: the colleague couldn't set sitter availability because that screen had no day/time control at all
Whole loop (2.7)	Compiled everything into one write-up, had Claude critique it as a product lead would, revised it, and added a feasibility check against the brief's budget and timeline.	<code>Brief-A-PawPal-Process.md</code>

What I learned, step by step

- 1. The process catches what instinct misses.** My first clustering dropped the two richest insights. My first two POV drafts named a solution ("a dog walker", "automated technology") instead of the need, and one claimed a timeframe the research didn't support.
- 2. HMW traps.** One of my HMW questions was a *solution in disguise* ("gradually reduce the dog's need for the owner" already picks a strategy). Another was too broad to act on.
- 3. Claude is most useful at choosing, not generating.** Five of my eight sketches drifted into general pet-care features instead of answering the HMW question. My first-choice third direction (monitoring) reduced the owner's risk but didn't build the dog's sense of security, so I swapped it.
- 4. Choose fidelity by the riskiest assumption,** not by what's easiest to build.
- 5. Following my own spec literally isn't the same as good judgment.** I built the wireframe strictly to my own element list, so ratings and reviews, the brief's number one concern, never appeared on any screen. I also initially treated a design critique as if it were a usability test.
- 6. Stating limitations honestly.** The write-up says plainly that each finding comes from one person ($n = 1$), and that the wireframe I tested was a generic booking flow rather than any of the three chosen directions.

7. Module 3 · Working with Designers (3 tasks, complete)

Why: as a PM I brief designers and review their work rather than design myself. "I don't like it" wastes a designer's afternoon; "the hierarchy doesn't match what the user came here to do" gives them something to act on. The module builds enough visual vocabulary to give that kind of feedback, and deliberately stops there.

How I worked, and what I produced

Task	What I did	What I produced
3.1 Visual literacy	Adopted five checks for any screen, critiqued a deliberately bad settings page Claude described, then critiqued Notion's real login page	Two critiques, each checked by Claude for where I'd stated preference as principle
3.2 Daily critique ritual	Critiqued ten real screens (five I like, five I don't) on six fields: user goal, hierarchy, type, colour, spacing, and which usability heuristics are violated	Ten structured critiques plus a count of taste-vs-principle slips
3.3 Design systems	Explored Google's Material 3 and Apple's Human Interface Guidelines and compared how each documents a button	Three sentences on what a design system buys a team, and notes on what a designer is protecting when they push back

My critique method

```

flowchart LR
    A["What is the<br/>user's goal?"] --> B["Run 5 checks:<br/>hierarchy, spacing,<br/>alignment, contrast,<br/>primary action"]
    B --> C["Map to a named<br/>heuristic"]
    C --> D{"Does the heuristic's<br/>real definition fit?"}
    D -- yes --> E["Principle:<br/>state it"]
    D -- no --> F["Taste:<br/>label it as preference"]
  
```

What I learned, step by step

- Terms I mixed up.** *Hierarchy* is what's most prominent and in what order; *alignment* is how elements line up on a shared axis. Centring everything on a page is consistent alignment, not a violation. I had labelled it one.

2. **My Notion critique:** one solid principle-based catch (a very light input border is an accessibility concern), two real observations filed under the wrong principle, and two personal preferences dressed up as principles.
3. **Where taste sneaks in.** Across the ten screens I found two clear slips: citing "user control and freedom" to justify wanting a dark-mode toggle (that heuristic is about undo and escape routes), and stretching "functional colour" to cover brand feel. Both happened when the critique format demanded a heuristic and my observation didn't fit one. The fix is to read the heuristic's definition before writing the sentence.
4. **What a design system buys, and costs.** Material specifies every button variant in every state because a Material app has no default look. Apple only documents the exceptions because iOS already supplies strong defaults. Departing from a system costs exactly what it was buying you, which is why "just make this button bigger" is rarely as small as it sounds.
5. **When a designer says "off-pattern",** they're protecting the token system, consistency across screens, their future maintenance load, or a decision already settled. It's reasonable to push back when there's evidence of a real user need, when the request keeps recurring, or when I'm willing to own the trade-off.

8. Module 4 · From Idea to Screen (10 tasks, complete)

Why: this is where requirements stop being words and become something people can click. The module covers structuring information, reading enough front-end code to talk with engineers, and describing a screen precisely enough that the first build comes back close to what I meant. Most of it uses **Brief B (Harvest Hub)**.

How I worked, and what I produced

Task	What I did	What I produced
4.1 Wireframing	Defined what a wireframe must contain and must leave out, then reduced three finished screens back to wireframes	Three reverse-engineered wireframes

Task	What I did	What I produced
4.2 Sketching by hand	Marked this Blocked for two days because I had no drawing pad, rather than faking it digitally. Then did 16 timed sketches (a claims list, a settings page)	Two annotated redraws
4.3 Information architecture and flows	Listed every piece of content and function Harvest Hub needs, grouped them into a sitemap, and drew the most important user flow	A 41-item inventory, a role-based sitemap (3 revisions) and the order-to-delivery flow
4.4 Dashboards	Wrote down dashboard anatomy and the six problems of dense interfaces, then checked them against real screens	Dense-Interface-Analysis.docx
4.5 Reading the front end	Learned basic HTML/CSS, read my own prototype's source code, and asked which changes are cheap vs expensive	Found a real bug by reading the code
4.6 First Claude Design prototype	Built a Settings screen and refined it over 10 rounds	A round log and the "protect the core requirement" prompt
4.7 States and edge cases	Designed 8 of the 11 states a data screen can be in, with real copy	A copy review
4.8 A real screen	Wireframed, built and self-reviewed Harvest Hub's Order Management screen	Three fixes
4.9 Accessibility	Audited that screen for seven accessibility responsibilities and fixed what failed	A fixed build, verified in the code

Task	What I did	What I produced
4.10 Precise specs	Described an unbuilt screen in ten lines, generated it with no refinement, found the gaps, rewrote and regenerated	Three versions of PawPal's Sitter Verification Queue

Harvest Hub sitemap (Task 4.3). I organised it by **role** instead of by feature, because each role comes to the product for a different job. Drivers deliberately have no login.

```

flowchart TD
    L["Login"] --> O["Ops team home"]
    L --> W["Warehouse home"]
    L --> OW["Owner home"]
    O --> OD["Orders and deliveries<br/>late count always visible"]
    OD --> N["New order /<br/>manual entry"]
    OD --> DT["Order detail<br/>incl. stock check"]
    OD --> DW["Driver workload"]
    W --> INV["Inventory"]
    INV --> SI["Stock item detail<br/>history tab"]
    OW --> BS["Business summary"]

```

The order-to-delivery flow (Task 4.3). The brief says a late order is "often worthless", so this is the most important task in the product. I revised it so every error path ends in a concrete outcome instead of a dead end.

```

flowchart TD
    A["New order received"] --> B{"Stock available?"}
    B -- no --> B2["Cancel, or Hold<br/>with timer"]
    B -- yes --> C["Reserve stock"]
    C --> D{"Driver available?"}
    D -- no --> D2["Reassign, or Hold<br/>with timer"]
    D -- yes --> E["Dispatch"]
    E --> F{"Edited after<br/>dispatch?"}
    F -- yes --> G["Re-verify"]
    G --> E
    F -- no --> H["In transit"]
    H --> I{"Delivered on time?"}
    I -- no --> J["Flag on<br/>owner dashboard"]
    I -- yes --> K["Mark delivered"]

```

What I learned, step by step

1. **A wireframe withholds** final colour, real imagery and polished type so the review stays about structure. My first definitions just repeated the labels I'd been given instead of defining them.
2. **Organise by what the user needs, not by the system's structure.** Claude pointed out that driver workload sat on a separate screen, a click away from the "reassign" decision it informs. In Task 4.8 I put it right beside the Assign Driver control.
3. **Dense interfaces need to be fast to read, not "clean".** For Harvest Hub, keyboard use and bulk actions are the same problem: the ops team is faster in their spreadsheet, so a new tool that only supports mouse clicks would be slower than what it replaces.
4. **Some fixes look cheap and aren't.** Reading my prototype's code, I found that both buttons in the Log-out confirmation called "cancel", so the real log-out never ran. It looked like a one-word fix, but the actual sign-out logic didn't exist anywhere.
5. **Requirements erode quietly.** Over 10 rounds, two separately reasonable fixes together removed two of the three things that made Log out visually distinct. So I now restate the core requirement and check it every round.
6. **Unhappy paths and states.** The 11 states: first-use empty, cleared empty, loading, partial load, error, no results, permission denied, offline, too much data, one item, a thousand items. Error copy must say what happened **and** what to do next; my Offline and One-item states failed that test.
7. **Accessibility is decided at design time:** contrast of at least 4.5:1 for body text, touch targets of 44–48 px, a logical focus order, a visible focus outline, never colour alone, clear labels, text alternatives. Focus order and missing labels on icon-only buttons can't be seen in a static design. I confirmed my fixes by reading the generated code rather than trusting how it looked.
8. **A requirement only gets built if it's in the spec's actual words.** A bulk action I'd named one step earlier never made it into my ten lines, so it wasn't built. Each rewrite then changed exactly what its words changed, no more and no less.

9. Module 5 · Delivery (3 of 13 tasks, in progress)

Why: delivery is the operational core of the job: planning, sprints, working with engineering and QA, and turning requirements into shipped work. This module runs on Harvest Hub.

Done so far

Task	What I did	Key learning
5.1 Project management basics	Drew the iron triangle and defined milestone, dependency, critical path, stakeholder and risk register, then asked how each shows up in day-to-day PM work	Iron triangle: scope, time and cost are linked, so changing one forces a change in another. Critical path: the <i>longest</i> chain of dependent tasks, which sets the minimum project duration. My first definition had it backwards as the fastest path. Dependency is the concept I expect to matter most in my first project.
5.2 Agile and Scrum	Wrote down every ceremony (purpose, attendees, time-box), the three roles, the three artefacts, and what a PM owns and asks in each	A PM's weekly effort goes mostly into backlog refinement : sharpening acceptance criteria, splitting oversized items, cutting stale ones
5.3 Software development lifecycle	Drew the SDLC phases, marked where product work happens, and compared waterfall with Agile	Marked done in the tracker; the write-up is still to be added

A Scrum sprint, as I understand it. Time-boxes are the maximum for a one-month sprint.

flowchart LR

```

PB["Product Backlog<br/>owned by PO"] --> SP["Sprint Planning<br/>max 8h"]
SP --> SB["Sprint Backlog<br/>owned by devs"]
SB --> DS["Daily Standup<br/>15 min"]
DS --> DS
DS --> INC["Increment<br/>meets Definition of Done"]
INC --> SR["Sprint Review<br/>max 4h, with stakeholders"]
SR --> RE["Retrospective<br/>max 3h, team only"]
RE --> PB
BR["Backlog Refinement<br/>ongoing, ~10% of capacity"] -.-> PB

```

Roles: the Product Owner owns *what and why*; the Scrum Master owns how well the team uses Scrum (a coach, not a boss); Developers own *how*. **Artefacts and what each commits to:** Product Backlog → Product Goal; Sprint Backlog → Sprint Goal; Increment → Definition of Done.

Next up: 5.4 build a Harvest Hub Jira board (an epic, five stories, three tasks) and learn to read a sprint board at a glance · 5.5 testing types, writing bug reports against my Task 4.8 prototype, bug vs change request · then APIs, requirements elicitation, user stories and acceptance criteria, estimation, a roadmap, status writing, leadership and stakeholder meetings.

10. Patterns in how I learn

Reading back through my journal, the same few habits come up again and again. I'm listing them openly because they're what I most want your help with.

Recurring habit	Where it showed up	The fix I'm practising
Spotting a loose end, then moving on without closing it	Two flagged HMW questions left unrewritten; unresolved notes on my sketches; Flexbox Froggy left half-done	Resolve it on the spot, or write down who closes it and when
Checking after the work instead of during it	Judging criteria written after seeing the output (1.4); requirement erosion caught only after 10 rounds (4.6); error copy audited after all eight states were built (4.7)	Set the check before starting and run it every round
Following my own spec literally instead of using judgment	Ratings and reviews missing from the PawPal wireframe; a bulk action lost between two steps (4.10)	Check the build against the user's need, not just my own list
Presenting taste as principle	Notion critique; the Calculator dark-mode and Savana colour critiques	Read the heuristic's real definition before writing the sentence
Organising by the system's structure instead of the user's	Driver workload a click away from the reassign decision	Ask "what does the person need visible right now?" while drafting

Recurring habit	Where it showed up	The fix I'm practising
Marking steps done from memory	Task 4.8 steps; Flexbox Froggy	Re-open the work before ticking the step

What's working well

- **Honest status.** I marked a task Blocked rather than faking it, and I record known limitations instead of hiding them.
- **Verifying against the source.** I check claims against primary sources, and I read the generated code to confirm my prototype fixes were really there.
- **Turning mistakes into reusable tools.** For example, the "protect the core requirement" prompt came directly out of watching the Log-out requirement erode.
- **Iterating visibly.** Most artefacts keep a revision history showing what changed and why.

11. What I'd like from you

Questions I haven't been able to resolve on my own

1. When a designer calls something "off-pattern", how do I tell quickly whether they're protecting the token system, consistency across screens, or a decision that's already been made?
2. In the Harvest Hub flow, what should happen when a "Hold (timer)" runs out? Is that a separate error path I need to design, or is "ops checks it manually" enough at this stage?
3. Should reading a prototype's code become a standard check before I treat any prototype as finished?
4. When should I merge near-duplicate prompts in my library, and when should I keep both?
5. Who on our account has admin rights to change org-level tool settings?

Asks

- ☐ Access to a Jira sandbox for Task 5.4, if one exists for the team
- ☐ A 20-minute live requirements-gathering session with you playing the Harvest Hub warehouse manager (Task 5.7)
- ☐ Your view on my recurring habits in Section 10: which one matters most to fix before live project work?

Glossary

Term	Meaning, in one line
Artefact	Something produced by a task, such as a critique, spec or prototype
Autoregressive generation	A language model producing text one token at a time, each predicted from what came before
Claude Design	Anthropic's tool for generating and refining interactive screen prototypes from written descriptions
Critical path	The longest chain of dependent tasks; it sets the shortest possible project duration
Dark pattern	Interface design that manipulates users against their interests
Design system	A shared library of components, tokens and rules that keeps a product consistent
Fidelity	How finished a prototype is, from paper sketch to high-fidelity interactive
Heuristic (Nielsen's ten)	Widely used rules of thumb for judging usability, e.g. "visibility of system status"
HMW (How Might We)	A problem reframed as an open question that invites ideas
Information architecture	How a product's content and functions are organised and labelled
Iron triangle	Scope, time and cost: changing one forces a change in another
n = 1	A finding based on a single person, so not yet a validated pattern
POV statement	A "[user] needs [need] because [insight]" problem statement
Prompt skeleton	Role · Context · Task · Format · Constraints

Term	Meaning, in one line
Scrum	An Agile framework of fixed-length sprints with defined roles, events and artefacts
Token (AI)	A chunk of text (word, part of a word, punctuation) that a model reads and writes
Token (design)	A named design value, such as a colour or spacing size, reused across a system
Usability test	Watching a real person attempt goal-based tasks and noting where they struggle
Wireframe	A structure-only layout of a screen, without final colour, imagery or polished type
XY problem	Asking about the fix you tried instead of the problem you actually have